Implement echo client server using TCP / UDP sockets.

Exp. no: 12.a

Date: 24/09/26

Aim:

To implement echo client server using TCP / UDP sockets.

Algorithm:

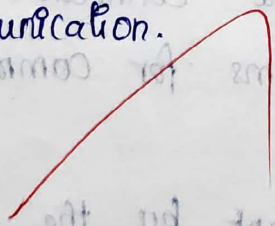
(i) TCP Echo Server and client:

* Start the TCP echo server and set the server port to the given port or use 5000 by default.
* Create a server socket and listen on the specified port.
* Wait for the client connection and create input and output streams for communication with client.
* Read a message sent by the client and display the received message and send it back to the client with "Echo:" added.
* If the received message is bye, close connection, otherwise continue receiving messages.

(ii) UDP :

Echo Server and client :

* Start the UDP server and listen on port 86000.
* Start the UDP client and create a datagram packet.
* Read the message from the client and convert it into a datagram packet and send it to the server.
* The server receives the packet and extracts the message. The server then adds "Echo:" to the packet and sends back the response
* The client receives the echoed response. If the client sends bye, stop communication.



Scanned with OKEN Scanner

Tcp code :

TCP Echo Client

```java
import java.io.*;
import java.net.*;

public class TCPEchoClient {
    Run | Debug
    public static void main(String[] args) throws IOException {
        String host = args.length > 0 ? args[0] : "localhost";
        int port = args.length > 1 ? Integer.parseInt(args[1]) : 5000;

        try (Socket socket = new Socket(host, port);
             PrintWriter out = new PrintWriter(socket.getOutputStream(), autoFlush: true);
             BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
             BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in))) {

            System.out.println("Connected to TCP Echo Server at " + host + ":" + port);
            System.out.println(x: "Type messages to send (type 'bye' to quit):");

            String userInput;
            while ((userInput = stdIn.readLine()) != null) {
                out.println(userInput);
                String response = in.readLine();
                System.out.println("Server: " + response);
                if (userInput.equalsIgnoreCase(anotherString: "bye")) {
                    break;
                }
            }
        }
        System.out.println(x: "Connection closed.");
    }
}
```

TCP Echo Server

```java
import java.io.*;
import java.net.*;

public class TCPEchoServer {
    Run | Debug
    public static void main(String[] args) throws IOException {
        int port = args.length > 0 ? Integer.parseInt(args[0]) : 5000;

        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("TCP Echo Server listening on port " + port);

            while (true) {
                try (Socket clientSocket = serverSocket.accept()) {
                    System.out.println("Client connected: " + clientSocket.getRemoteSocketAddress());

                    BufferedReader in = new BufferedReader(
                            new InputStreamReader(clientSocket.getInputStream()));
                    PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), autoFlush: true);

                    String line;
                    while ((line = in.readLine()) != null) {
                        System.out.println("Received: " + line);
                        out.println("Echo: " + line);
                        if (line.equalsIgnoreCase(anotherString: "bye")) {
                            break;
                        }
                    }
                    System.out.println("Client disconnected: " + clientSocket.getRemoteSocketAddress());
                } catch (IOException e) {
                    System.out.println("Connection error: " + e.getMessage());
                }
            }
        }
    }
}
```

Scanned with OKEN Scanner

~~Output :~~ UDP Code :

UDP Echo Client

```java
import java.io.*;
import java.net.*;

public class UDPEchoClient {
    Run | Debug
    public static void main(String[] args) throws IOException {
        String host = args.length > 0 ? args[0] : "localhost";
        int port = args.length > 1 ? Integer.parseInt(args[1]) : 6000;

        InetAddress address = InetAddress.getByName(host);

        try (DatagramSocket socket = new DatagramSocket();
             BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in))) {

            socket.setSoTimeout(timeout: 5000); // avoid blocking forever if a reply is lost
            System.out.println("UDP Echo Client ready to send to " + host + ":" + port);
            System.out.println(x: "Type messages to send (type 'bye' to quit):");

            String userInput;
            while ((userInput = stdIn.readLine()) != null) {
                byte[] sendData = userInput.getBytes();
                DatagramPacket request = new DatagramPacket(sendData, sendData.length, address, port);
                socket.send(request);

                byte[] buffer = new byte[1024];
                DatagramPacket response = new DatagramPacket(buffer, buffer.length);
                try {
                    socket.receive(response);
                    String reply = new String(response.getData(), offset: 0, response.getLength());
                    System.out.println("Server: " + reply);
                } catch (SocketTimeoutException e) {
                    System.out.println(x: "No response received (timeout).");
                }

                if (userInput.equalsIgnoreCase(anotherString: "bye")) {
                    break;
                }
            }
        }
        System.out.println(x: "Client finished.");
    }
}
```

UDP Echo Server

```java
import java.net.*;

public class UDPEchoServer {
    Run | Debug
    public static void main(String[] args) throws Exception {
        int port = args.length > 0 ? Integer.parseInt(args[0]) : 6000;
        byte[] buffer = new byte[1024];

        try (DatagramSocket socket = new DatagramSocket(port)) {
            System.out.println("UDP Echo Server listening on port " + port);

            while (true) {
                DatagramPacket request = new DatagramPacket(buffer, buffer.length);
                socket.receive(request);

                String received = new String(request.getData(), offset: 0, request.getLength());
                InetAddress clientAddress = request.getAddress();
                int clientPort = request.getPort();
                System.out.println("Received from " + clientAddress + ":" + clientPort + " -> " + received);

                byte[] responseData = ("Echo: " + received).getBytes();
                DatagramPacket response = new DatagramPacket(
                        responseData, responseData.length, clientAddress, clientPort);
                socket.send(response);
            }
        }
    }
}
```

Scanned with OKEN Scanner

Output for TCP :

ECHO CLIENT

```
devyesh@Macs-MacBook-Air Downloads % javac TCPEchoClient.java
devyesh@Macs-MacBook-Air Downloads % java TCPEchoClient
Connected to TCP Echo Server at localhost:5000
Type messages to send (type 'bye' to quit):
Hello!
Server: Echo: Hello!
This is Echo Client, Do you copy ?
Server: Echo: This is Echo Client, Do you copy ?
Hello!
Server: Echo: Hello!
```

ECHO SERVER

```
devyesh@Macs-MacBook-Air Downloads % javac TCPEchoServer.java
devyesh@Macs-MacBook-Air Downloads % java TCPEchoServer
TCP Echo Server listening on port 5000
Client connected: /127.0.0.1:49419
Received: Hello!
Received: This is Echo Client, Do you copy ?
Received: Hello!
```

Output for UDP :

ECHO CLIENT

```
devyesh@Macs-MacBook-Air Downloads % javac UDPEchoClient.java
devyesh@Macs-MacBook-Air Downloads % java UDPEchoClient
UDP Echo Client ready to send to localhost:6000
Type messages to send (type 'bye' to quit):
Hello!
Server: Echo: Hello!
Request permission to authorize
Server: Echo: Request permission to authorize
Permission granted
Server: Echo: Permission granted
```

ECHO SERVER

```
devyesh@Macs-MacBook-Air Downloads % javac UDPEchoServer.java
devyesh@Macs-MacBook-Air Downloads % java UDPEchoServer
UDP Echo Server listening on port 6000
Received from /127.0.0.1:50050 -> Hello!
Received from /127.0.0.1:50050 -> Request permission to authorize
Received from /127.0.0.1:50050 -> Permission granted
Received from /127.0.0.1:50050 -> quit
Received from /127.0.0.1:50050 -> bye
```

Result :

Thus, the echo client server using TCP/UDP sockets has been successfully implemented and the output was verified.

Scanned with OKEN Scanner